

CS 550 Operating Systems

Spring 2025

Dynamic relocation

Memory virtualization overview

- Mechanism of **CPU virtualization**
 - **Efficiency** and **control** are the two main goals of modern OSes.
 - Let program run directed on hardware (for efficiency)
 - At certain key points in time (*e.g.*, system call, interrupt), OS retains control over hardware for efficient virtualization
 - Mechanism of **memory virtualization**
 - **Efficiency**: via hardware support
 - **Control**: the OS ensures that no application is allowed to access any memory but its own.
 - via hardware support too
 - **Flexibility**: make system easier to program and to use address space
- The general technique → **(hardware-based) address translation**

Address translation

- Hardware does [virtual address (provided by instruction) → physical address]
- Hardware alone cannot virtualize memory. OS should also help to set up hardware, keeps track of free/used memory, and maintains control over how memory is used.
 - to create illusion that each program has its own **private** memory, even though many programs are sharing memory at the same time
- Create useful, powerful, and easy to use abstraction

Address translation

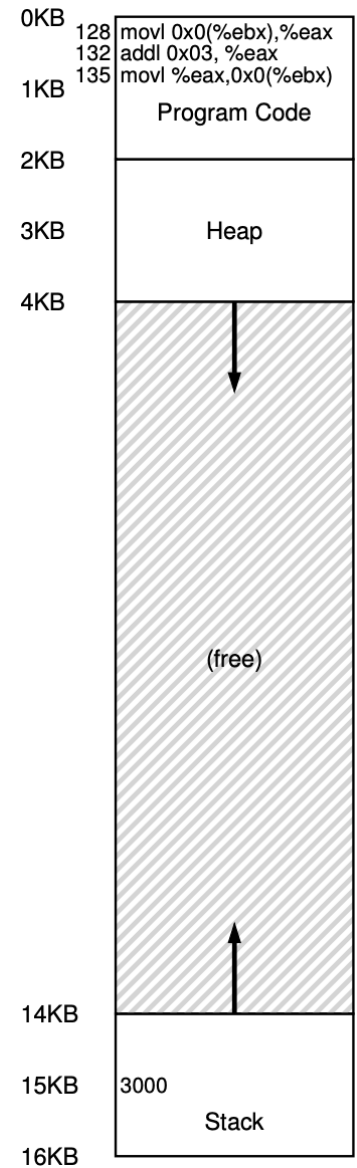
- Different techniques on address translation
 - Base and bounds
 - Segmentation
 - Paging
- Start from a simple one: base and bounds.
 - Assumptions (will be relaxed gradually)
 - user's address space must be placed **contiguously** in physical memory;
 - its size is less than the size of physical memory;
 - each address space is exactly the same size
- An example to understand the need of address translation

```
void func() {
    int x = 3000;
    x = x + 3;
```

↓ compiler

```
128: movl 0x0(%ebx), %eax    ;load 0+ebx into eax
132: addl $0x03, %eax       ;add 3 to eax register
135: movl %eax, 0x0(%ebx)    ;store eax back to mem
```

- When these instructions run, *from the perspective of the process*, the following memory accesses take place.
 - Fetch instruction at address 128
 - Execute this instruction (load from address 15 KB)
 - Fetch instruction at address 132
 - Execute this instruction (no memory reference)
 - Fetch the instruction at address 135
 - Execute this instruction (store to address 15 KB)

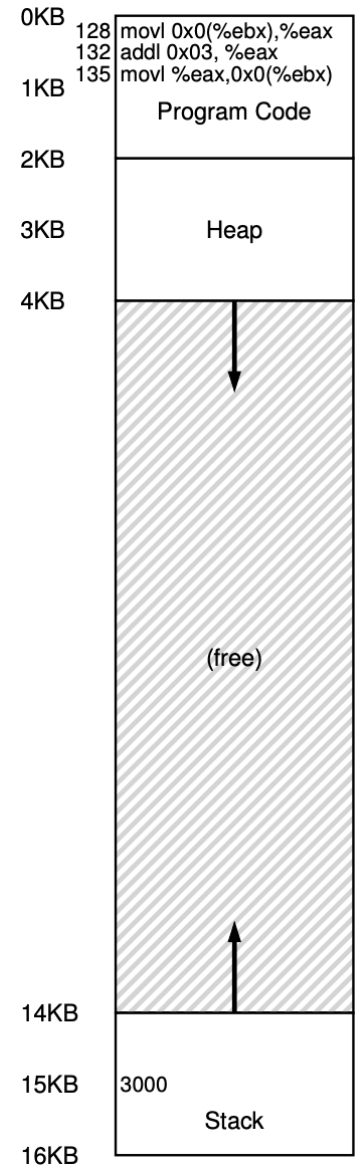


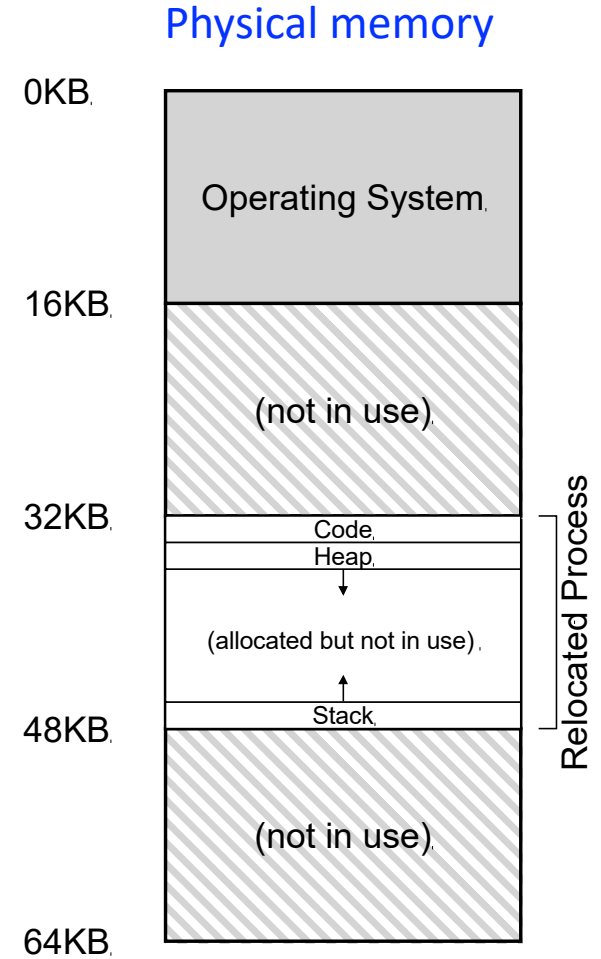
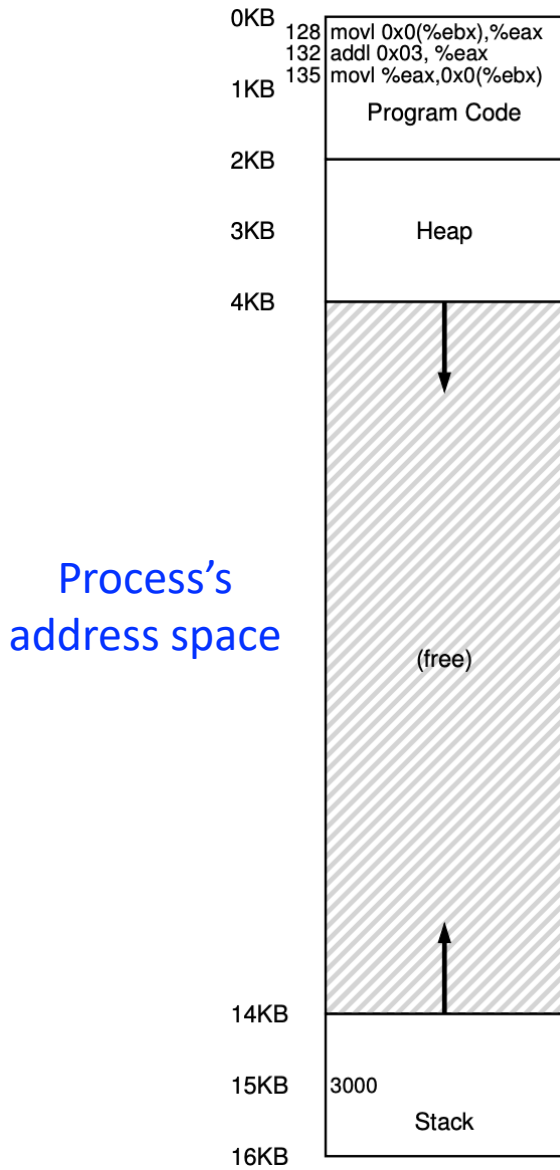
```
void func() {
    int x = 3000;
    x = x + 3;
```

↓ compiler

```
128: movl 0x0(%ebx), %eax    ;load 0+ebx into eax
132: addl $0x03, %eax        ;add 3 to eax register
135: movl %eax, 0x0(%ebx)    ;store eax back to mem
```

- Address space starts from address 0
- To virtualize memory, OS needs to place the process somewhere else in physical memory
- Problem: how can OS relocate process in memory that is **transparent** to the process?
- Solution: hardware **interposes** on each memory access, and translates each virtual address issued by the process to a physical address where the desired information is actually stored





Dynamic (HW-based) relocation

- The first address translation technique we discuss.
- Two registers
 - Base register
 - Bound register (a.k.a. limit register)
 - The two registers allow us to place the address space anywhere we'd like in physical memory, while ensuring that the process can only access its own address space.
- The relocation process
 - Each program is written and compiled as if it is loaded at address 0
 - When a program starts running, OS decides where in physical memory it should be loaded and sets the **base register** to that value (for example, the base register is set to 32KB for the previous example)

- Each memory reference generated by the process is a **virtual address**; the hardware in turn adds the contents of the base register to this address and the result is a physical address that can be issued to the memory system.

$$\text{physical address} = \text{virtual address} + \text{base}$$

- The bounds register is for protection
 - In the previous example, the bounds register would always be set to 16 KB. If a process generates a virtual address that is greater than the bounds, CPU will raise an exception.
- The base and bounds registers are hardware structures kept on the chip (one pair per CPU).
 - The part of the processor that helps with address translation is called the memory management unit (MMU)
 - More sophisticated memory management techniques later, which need adding more circuits to the MMU.

Dynamic (HW-based) relocation

Base and bound pseudo-code:

1. $\text{PhysAddr} = \text{VirtualAddress} + \text{BASE}$
2. **if** ($\text{PhysAddr} \geq \text{BASE} + \text{BOUND}$)
3. $\text{RaiseException}(\text{PROTECTION_FAULT})$
4. $\text{Register} = \text{AccessMemory}(\text{PhysAddr})$

Dynamic relocation (base and bound) example

Imagine a process has been loaded to physical memory and the base and bound register have been set to ➔

Base = 16 KB (16,384)

Bound (limit) = 4 KB (4,096)

Base + Bound = 20KB (20480)

Virtual address (memory references made by the process)	Physical address?	Valid access?
0		
1 KB		
3000		
4096		
4400		

Dynamic relocation (base and bound) example

Imagine a process has been loaded to physical memory and the base and bound register have been set to ➔

Base = 16 KB (16,384)

Bound (limit) = 4 KB (4,096)

Base + Bound = 20KB (20480)

Virtual address (memory references made by the process)	Physical address?	
0	16 KB (16384)	16 KB < 20 KB ➔ Valid access
1 KB	17 KB	17 KB < 20 KB ➔ Valid access
3000	$16384 + 3000 = 19384$	$19384 < 20480$ ➔ Valid access
4096	$16384 + 4096 = 20480$	$20480 \geq 20480$ ➔ Exception (out of bound)
4400	$16384 + 4400 = 20784$	$20784 \geq 20480$ ➔ Exception (out of bound)

Static (S/W-based) relocation

- Static relocation is preformed by the **loader** when an executable is loaded to memory
 - take an executable that is about to be run and rewrites its addresses to the desired **offset** in physical memory
 - *e.g.*, `movl 1000,%eax`, and the address space of the program was loaded starting at address 3000 (and not 0, as the program thinks), the loader would rewrite the instruction to **offset** each address by 3000 (`movl 4000,%eax`).

Advantages of Base and Bound

- Simple hardware implementation
- Simple to manage each process' virtual space
- Processes can be loaded at arbitrary fixed addresses
- Offers protection and isolation
- Offers flexible placement of data in memory

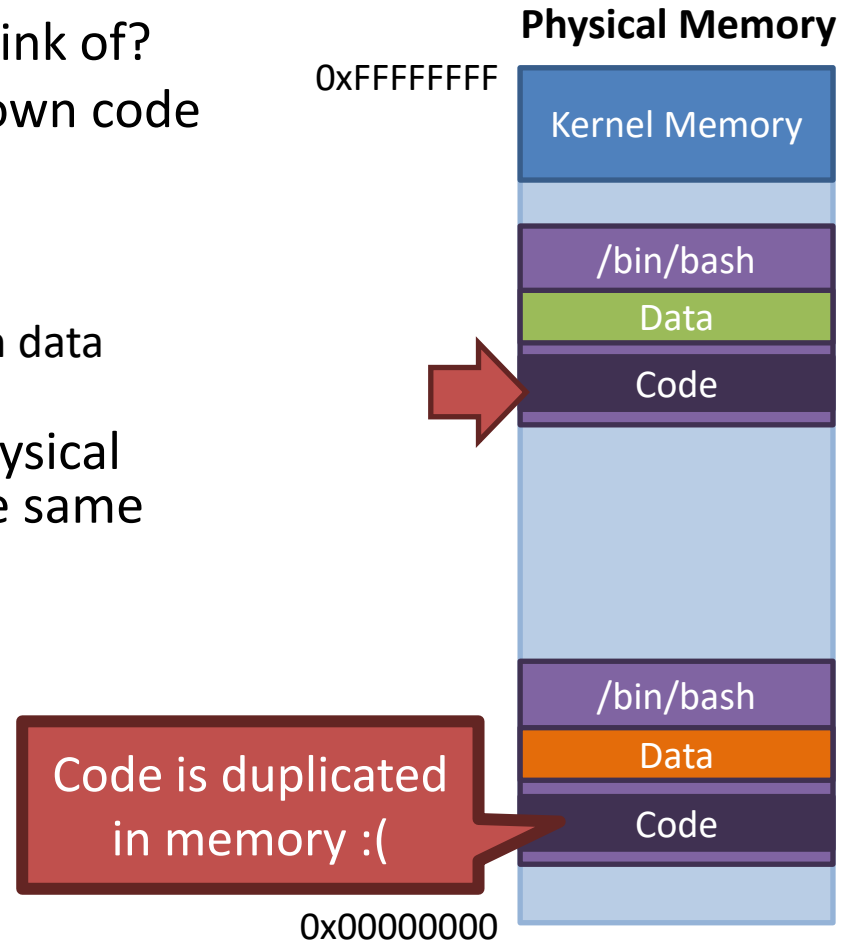
Limitations with dynamic relocation

- OS has to take care of many things (low efficiency, compared to earlier system where fixed slots of allocation was used)
 - OS must take action when a process is created, finding space for its address space in memory
 - Works fine given the assumptions that each address space is (1) smaller than the size of physical memory and (2) the same size.
 - OS must take action when a process is terminated, reclaiming all of its memory for use in other processes or the OS
 - OS must also take action when a context switch occurs (save/restore base-bounds register values to/from PCB)

Limitations with dynamic relocation

What other limitations can you think of?

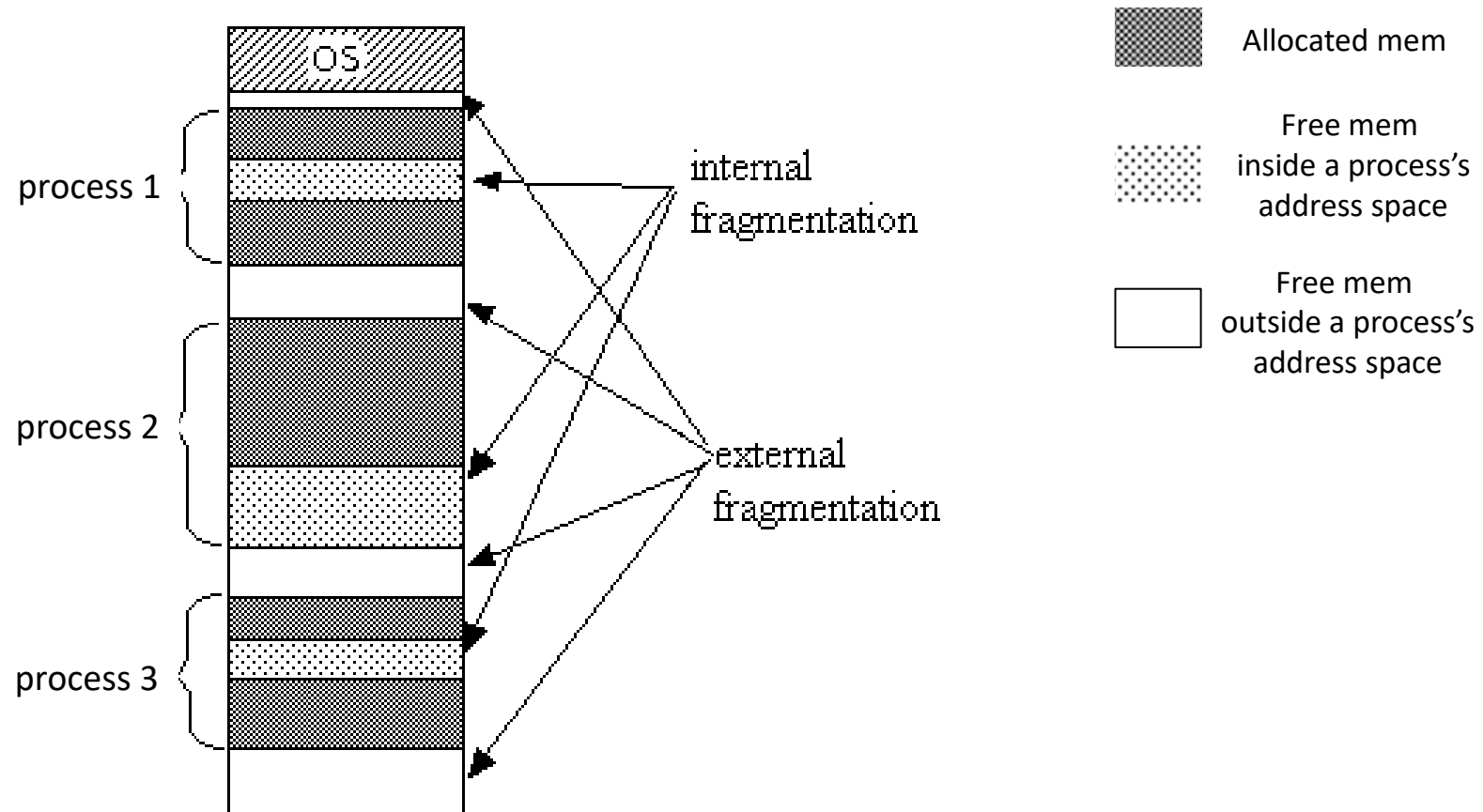
- Processes can overwrite their own code
 - Processes aren't protected from themselves
- No sharing of memory
 - Code (read-only) is mixed in with data (read/write)
- Process does not use all the physical memory allocated (which is the same size as the address space).
 - Internal fragmentation



Fragmentation

- Fragmentation is a phenomenon in which storage space is used inefficiently, reducing capacity or performance and often both.
- Types of fragmentation
 - **Internal fragmentation**: the wasted space within each allocated block.
 - **External fragmentation**: the various free spaced holes that are generated in either your memory or disk space.

Internal and external fragmentation



Summary

- Address translation
 - efficiency via hardware
 - transparent
- Base and bounds: protection
- Limitations
- Solution: **segmentation**

